KU Leuven

Project Report

Bicycle Traffic Monitoring and Forecasting

Modern Data Analytics

Group 16

Ghasemian Moghaddam Alireza	1079987
Xuanang Li	r1070796
Yavuz Tok	r0882205
Zeren Su	r1082963

Academic year 2025–2026

1 Executive Summary

This project develops a Python Shiny dashboard and supporting data pipeline for station-level bicycle traffic monitoring and short-term forecasting in Flanders. The data comes from the public AWV *fietstellingen* dataset, which provides monthly automatic count files together with site and direction metadata. The resulting system is designed for transport planners and infrastructure managers who need to monitor bicycle usage, identify pressure points, and decide which stations deserve further investigation.

The final implementation is not a one-off notebook analysis. It is structured as a small analytics system: raw monthly CSV files are ingested, filtered to cyclist counts, aggregated to hourly station-level observations, stored as Parquet files, evaluated with repeatable model backtests, and served through an interactive dashboard. The pipeline currently processes 81 monthly files from August 2019 to April 2026, producing 5,311,003 hourly station records for 150 stations. Across the full processed dataset, the total number of counted cyclists is 107,385,663. Median station data coverage is 99.58%, which makes the data broadly suitable for monitoring while still requiring quality checks for some stations.

For forecasting, we compare three model families: a seasonal naive baseline, a feature-based histogram gradient boosting model, and Prophet. The saved forecasting outputs are generated for the 12 busiest eligible stations by default, while monitoring and planning pages continue to use all 150 stations. This choice keeps the demonstration responsive and computationally feasible, while the code supports an `-all-stations` option for full forecasting runs. Model performance is evaluated with three rolling backtest windows, each using a 168-hour test horizon. Based on average WAPE across these backtests, histogram gradient boosting is the best model for 9 of the 12 forecasted stations, seasonal naive is best for 2 stations, and Prophet is best for 1 station. The median WAPE values are 24.97% for histogram gradient boosting, 30.14% for seasonal naive, and 38.26% for Prophet.

The dashboard contains five views: an overview map, station detail analysis, forecasting, planning insights, and ML engineering status. The planning view converts historical demand, peak pressure, recent growth, forecast reliability, and data coverage into an infrastructure priority score. The ML engineering view exposes the generated artifacts, model run status, and re-run commands, making the project reproducible and easier to maintain.

2 Problem Context and Objectives

The Agentschap Wegen en Verkeer (AWV) manages major road and cycling infrastructure in Flanders. Its automatic bicycle counting stations provide a valuable source for understanding how cycling demand changes across locations and over time. This information supports planning decisions such as where to monitor infrastructure pressure, where demand is growing, and where data quality may be too weak to support investment decisions.

The project is deliberately framed at the station level. The available data describes fixed counting locations rather than the entire cycling network. Therefore, the dashboard should not be interpreted as a live routing or real-time traffic application for cyclists. Instead, its main purpose is to support planning and monitoring questions for infrastructure stakeholders:

- Which counting stations have the highest bicycle traffic?
- Which stations show strong recent growth or high peak-hour pressure?
- Which stations have sufficiently reliable data for analysis?
- Can short-term hourly demand be forecasted for selected high-traffic stations?

- How can the data and forecasting workflow be rerun when new monthly files are released?

These questions also address the feedback received on the project proposal. The initial proposal clearly identified dashboard stakeholders, but the proposed design risked being too limited for planning and too close to a one-off exercise. The implemented system therefore emphasizes three improvements: a repeatable data pipeline, explicit model evaluation and retraining logic, and planning-oriented dashboard indicators.

3 Data Source and Preprocessing

3.1 Input Data

The AWV dataset consists of monthly count files and metadata files. The count files contain observations such as the station ID, direction, detected object type, start and end time, and count. Since the project focuses on bicycle infrastructure, only records with type `FIETSERS` are retained. The `sites.csv` metadata file provides station names, municipalities, coordinates, and installation dates. Direction metadata is available in `richtingen.csv`, although the current station-level dashboard aggregates directions into total station counts for interpretability.

Component	Use in this project
`data-YYYY-MM.csv`	Monthly raw counts; filtered to cyclist records and aggregated to hourly station totals.
`sites.csv`	Station metadata, including coordinates, municipality, and station labels.
`richtingen.csv`	Direction metadata; used to understand the source structure, while the dashboard currently reports station-level totals.

Table 1: Main input files from the AWV bicycle count dataset.

3.2 Preprocessing Pipeline

The preprocessing script reads all monthly files in chunks to avoid loading the raw CSV dataset into memory at once. The main preprocessing steps are:

1. Read monthly count files with explicit column names.
2. Filter to cyclist observations only.
3. Parse timestamps and round observations to hourly level.
4. Aggregate counts by `site_id` and hour.
5. Merge station-level metadata from `sites.csv`.
6. Compute station summary features, including average daily traffic, recent growth, peak-hour indicators, and data coverage.
7. Store processed data as Parquet and CSV artifacts for fast dashboard loading.

The dashboard does not read the raw CSV files directly. This design keeps the app responsive and separates batch processing from interactive visualization. It also makes the pipeline more reproducible: the raw data, processed data, model outputs, and dashboard each have clear responsibilities.

3.3 Data Coverage

Data coverage is computed for each station as:

Pipeline statistic	Value
Monthly files processed	81
Date range	2019-08-01 to 2026-04-30
Hourly station records	5,311,003
Stations with counts	150
Total counted cyclists	107,385,663
Mean station coverage	98.86%
Median station coverage	99.58%

Table 2: Processed dataset summary.

$$\text{coverage rate} = \frac{\text{observed hourly records}}{\text{expected hourly records between first and last observation}}.$$

This metric measures whether a station has records for the hours in which it was active. A zero count still counts as observed data; a completely missing hour does not. Coverage is used both as a quality indicator and as a planning safeguard. A station with low coverage may appear to have low demand or unusual growth simply because the sensor was missing records. For this reason, the planning logic flags low-coverage stations for sensor or data quality checks before using them for infrastructure investment decisions.

4 System Architecture

The implemented project follows a modular analytics architecture. The main artifacts are stored in `data/processed/`, while source scripts are stored in `codes/` and the dashboard is stored in `app/`. The raw data directory and processed Parquet files are ignored by Git to avoid uploading large data files to GitHub.

Artifact	Role
`hourly_counts.parquet`	Hourly station-level cyclist counts used by dashboard views.
`station_summary.parquet`	Station-level features, metadata, coverage, growth, and planning indicators.
`model_evaluation.csv`	Average model metrics across rolling backtest windows.
`model_backtest_windows.csv`	Detailed per-window model evaluation results.
`forecast_outputs.parquet`	Saved future hourly forecasts for trained stations.
`pipeline_run_summary.json`	Metadata for preprocessing runs.
`forecast_run_summary.json`	Metadata for forecast training and evaluation runs.
`pipeline_orchestration.json`	Metadata for the one-command pipeline runner.

Table 3: Generated artifacts consumed by the dashboard.

The full workflow can be summarized as a sequence of reproducible stages:

Raw CSV files	Processed Parquet	Backtest metrics	Shiny dashboard
Monthly AWV counts	Hourly station features	Model evaluation and forecasts	Monitoring, forecasting, and planning views

The orchestration script `codes/run_pipeline.py` allows the pipeline to be rerun from the command line. For example, the command `python codes/run_pipeline.py -skip-preprocess -top-stations 12 -backtest-windows 3` retrains and evaluates the forecasting models with-

out rebuilding the processed dataset. This is important because the source data is updated monthly, and a realistic analytics system should support periodic refreshes.

5 Implementation and Course Alignment

The implementation was designed to reflect the main themes of the Modern Data Analytics course rather than only producing an attractive dashboard. The project combines Python data processing, machine learning pipelines, dashboarding, versioning, container readiness, and analytics infrastructure thinking.

Course theme	**How it appears in the project**
Python fundamentals	The pipeline is structured as reusable scripts with functions for preprocessing, model training, evaluation, and orchestration.
Data science packages	Pandas is used for chunked CSV ingestion, time parsing, aggregation, feature engineering, and metric computation.
Data science algorithms	The dashboard uses ranking, growth calculations, coverage metrics, percentile scores, and forecasting model comparisons.
Scikit-learn pipelines	Histogram gradient boosting is implemented as a scikit-learn pipeline with imputation and engineered temporal features.
Dashboarding in Python	Shiny for Python and Plotly are used to build interactive maps, time-series views, forecast views, and data tables.
Code collaboration and versioning	Large raw and generated data files are excluded through `.gitignore`, while source scripts and configuration files remain versionable.
Containerisation	A Dockerfile and `.dockerignore` are included so the dashboard can be containerized after processed data is mounted.
Analytics infrastructure	The ML Engineering page exposes pipeline status, generated artifacts, and re-run commands for operational transparency.

Table 4: Relationship between course concepts and project implementation.

This alignment is important because the project is not only evaluated as a data analysis exercise. It should also demonstrate an understanding of how a data product can be organized, rerun, and maintained. For that reason, the project includes both user-facing functionality and engineering-facing transparency.

5.1 Design Decisions

Several design decisions were made to keep the system robust:

- **Hourly aggregation**: the original data is often at 15-minute resolution, but hourly data is easier to model, faster to visualize, and sufficient for short-term planning.
- **Station-level totals**: directions are aggregated so that the dashboard answers station-level planning questions rather than direction-specific traffic questions.
- **Parquet storage**: processed tables are stored in Parquet because they load much faster than repeatedly parsing large CSV files.
- **Default forecast subset**: all stations are monitored, but the default forecast pipeline trains on the 12 busiest eligible stations to keep model training practical during development and presentation.
- **Explicit artifacts**: model metrics, forecast outputs, and pipeline summaries are saved as files so that the dashboard consumes stable outputs rather than retraining models interactively.

6 Forecasting Methodology

6.1 Forecasting Task

The forecasting task is short-term station-level prediction. For a selected station, the system predicts future bicycle counts at hourly resolution. The dashboard allows the user to view either the next 24 hours or the next 168 hours. The underlying saved forecast artifact is generated for 168 hours, equivalent to seven days of hourly predictions.

The default saved forecasts are trained for the 12 busiest eligible stations. This is a practical design choice: all 150 stations are used in the monitoring and planning views, but model training is more computationally expensive, especially for Prophet. The code supports `-all-stations` for complete forecast coverage, but the default dashboard demo focuses on the highest-demand stations to keep training time manageable.

6.2 Candidate Models

Three forecasting approaches are compared:

- **Seasonal naive baseline**: predicts using the same hour from the previous week, with fallback values based on hour-of-week medians.
- **Histogram gradient boosting**: a feature-based scikit-learn model using calendar features, lag features, and rolling statistics.
- **Prophet**: a time-series model with daily, weekly, and yearly seasonality where enough history is available.

The seasonal naive model is essential because cycling counts are strongly periodic. A more complex model only adds value if it can beat this simple weekly-pattern baseline. Histogram gradient boosting represents a flexible machine learning approach for tabular time features. Prophet represents a more classical time-series approach for trend and seasonality.

6.3 Feature Engineering

The histogram gradient boosting model uses only information available from the bicycle count history. This avoids the complexity and reliability risks of adding external weather or holiday sources at this stage. The engineered features include:

- cyclic hour-of-day features using sine and cosine transformations;
- cyclic weekday features to capture weekly commuting and leisure patterns;
- cyclic month features to capture broad seasonal variation;
- a weekend indicator;
- lag features at 1 hour, 24 hours, and 168 hours;
- rolling means over the previous 24 and 168 hours;
- a rolling 24-hour standard deviation to capture short-term volatility.

The 168-hour lag is especially important because the seasonal naive baseline is strong. By giving the machine learning model access to weekly lag information, the comparison is fairer: the machine learning model is not asked to rediscover weekly seasonality from calendar features alone. Missing lag values are handled through median imputation inside the scikit-learn pipeline.

6.4 Rolling Backtest Evaluation

Instead of evaluating only the last week once, the implemented pipeline uses rolling backtests. Each model is evaluated over three historical windows. Each window uses the latest available training history before the test period and predicts a 168-hour test horizon. The three windows are spaced 168 hours apart, so each window corresponds to a different held-out week.

For each station and model, the metrics are averaged across the rolling windows. The final reported metrics are MAE, RMSE, and WAPE:

$$\text{WAPE} = \frac{\sum_t |y_t - \hat{y}_t|}{\sum_t y_t} \times 100.$$

WAPE is especially useful for this project because it scales absolute error by total observed demand. This makes it easier to compare stations with different traffic volumes.

After evaluation, the system retrains models using the latest available history and generates future 168-hour forecasts. This separates model evaluation from future prediction: the backtests estimate expected performance, while the final forecast uses the most recent data.

6.5 Model Selection Policy

The dashboard does not assume one global best model for every station. Instead, it selects the best model per station based on average WAPE across the rolling backtests. This is appropriate because bicycle count stations differ in volume, regularity, local context, and sensor behavior. A station with a very stable weekly pattern may be best served by the seasonal naive baseline, while a station with nonlinear interactions between weekday, hour, and recent demand may benefit from histogram gradient boosting.

This policy also helps make the forecasting results interpretable. The model comparison table shows whether a candidate model improves on the seasonal naive baseline. If the baseline wins, that is not a failure; it means the weekly pattern is strong enough that a simple model is hard to beat. In a planning context, this honesty is valuable because it prevents overclaiming the performance of complex models.

7 Forecasting Results

The current run evaluates 12 selected stations, 3 models, and 3 rolling backtest windows. This produces 108 detailed backtest rows and 36 aggregated model evaluation rows. The future forecast artifact contains 6,048 rows, corresponding to 12 stations, 3 models, and 168 forecast hours.

Model	Mean WAPE	Median WAPE	Min WAPE	Max WAPE
Histogram gradient boosting	29.51	24.97	20.75	51.21
Seasonal naive	30.93	30.14	22.95	44.78
Prophet	38.37	38.26	31.95	46.01

Table 5: Model performance across selected forecast stations, averaged over rolling backtests.

Histogram gradient boosting is the best-performing model for 9 of the 12 forecasted stations. Seasonal naive remains best for 2 stations, and Prophet is best for 1 station. This result is important because it demonstrates the value of model comparison rather than assuming that a more complex model is automatically better. Prophet is useful to include as a time-series reference, but in this dataset it does not consistently outperform the simpler baseline or the feature-based machine learning model.

Best model by station	Number of stations
Histogram gradient boosting	9
Seasonal naive	2
Prophet	1

Table 6: Best model counts based on average WAPE.

The model results also show that the baseline is strong. This is expected for bicycle traffic because weekly patterns are often stable. The added value of histogram gradient boosting appears when nonlinear combinations of hour, weekday, month, recent lag values, and rolling averages improve the forecast. In contrast, Prophet tends to smooth the data and may underperform when station-level traffic has local irregularities or sharp short-term patterns.

8 Dashboard Design

The final dashboard is implemented with Shiny for Python and Plotly. It is organized into five pages, each corresponding to a different decision or monitoring task.

8.1 Overview

The overview page provides a spatial and temporal summary of bicycle traffic. Users can choose a date range and inspect the number of active stations, hourly rows, and total cyclist counts in that period. A map displays station locations, with marker size and color linked to average daily traffic. Ranking tables show the highest average daily traffic and the fastest recent growth.

This view is useful for identifying important stations and understanding the geographic distribution of cycling demand. It also gives a first indication of where planners may want to investigate further.

8.2 Station Detail

The station detail page focuses on one selected station. It shows average daily traffic, peak hour count, data coverage, daily traffic over time, average hourly patterns, and weekday profiles. This page supports exploratory analysis of local temporal patterns, such as morning and evening peaks or weekday-weekend differences.

8.3 Forecast

The forecasting page shows recent historical counts and saved future forecasts. The user can select a forecast horizon of 24 or 168 hours. The page reports model metrics from rolling backtests, the best model for the selected station, and whether candidate models improve over the seasonal naive baseline. A forecast scope card explains that monitoring uses all 150 stations, while saved advanced forecasts are currently trained for the selected 12 stations by default.

8.4 Planning Insights

The planning page converts descriptive statistics and model outputs into planning-oriented indicators. The main feature is an infrastructure priority shortlist. The priority score is defined as:

$$\text{Priority} = 0.35D + 0.25P + 0.20G + 0.10R + 0.10C,$$

where D is demand, P is peak pressure, G is recent growth, R is forecast reliability, and C is data coverage. These components are percentile-based or normalized indicators, so the score is designed for ranking and screening rather than direct cost-benefit analysis.

The dashboard also generates planning recommendations. For example, low-coverage stations are flagged for data quality checks, high-priority and high-growth stations are marked as capacity upgrade candidates, and high-demand stations are flagged for comfort and capacity review. This directly supports the planning use case described in the project information.

Station	Priority	Coverage	Recommendation
Nieuwpoort teller 1	88.69	1.00	Capacity upgrade candidate: high use and recent growth
Herent teller 2	88.46	1.00	Capacity upgrade candidate: high use and recent growth
Herentals totem	86.92	1.00	Capacity upgrade candidate: high use and recent growth
Nieuwpoort totem	86.32	1.00	Capacity upgrade candidate: high use and recent growth
As	85.79	1.00	Capacity upgrade candidate: high use and recent growth
Leuven totem	83.16	1.00	Capacity upgrade candidate: high use and recent growth

Table 7: Example top planning priority stations from the current run.

8.5 ML Engineering

The ML engineering page makes the system transparent. It shows the number of processed rows, trained models, forecast horizon, pipeline status, generated artifacts, and re-run commands. This page is intentionally included to demonstrate that the project is not just an interactive visualization. It documents the operational workflow from raw data to processed features, model evaluation, forecast outputs, and dashboard consumption.

8.6 Demonstration Flow

For presentation, the dashboard should be demonstrated in a fixed order. The recommended flow is:

1. **Overview**: introduce the spatial map, date range, total cyclist counts, high-demand stations, and recent growth.
2. **Station Detail**: select one busy station and explain daily trends, hourly patterns, weekday profiles, and data coverage.
3. **Forecast**: show the forecast scope, model comparison, rolling backtest metrics, and 24-hour versus 168-hour forecast horizons.
4. **Planning Insights**: explain the priority score, recommendation labels, high-demand stations, peak pressure stations, growth signals, and data quality risks.
5. **ML Engineering**: close by showing the generated artifacts and re-run commands, emphasizing that the dashboard is backed by a repeatable pipeline.

This flow is designed to move from descriptive monitoring to predictive modeling and then to operational transparency. It also helps communicate the project as a complete analytics system instead of a collection of unrelated plots.

9 Discussion

The project improves on the original proposal in several ways. First, it implements Parquet-based preprocessing and separates raw data ingestion from dashboard serving. Second, it extends the original baseline-versus-Prophet plan by adding a feature-based machine learning model. Third, it uses rolling backtests rather than a single train-test split. Fourth, it adds planning-oriented indicators that translate data patterns into actionable signals for infrastructure managers.

The results also highlight an important lesson: more complex models are not always better. The seasonal naive baseline is competitive because weekly cycling patterns are strong. Prophet is

interpretable and useful as a time-series benchmark, but it does not consistently win in this dataset. Histogram gradient boosting performs best for most forecasted stations, suggesting that nonlinear tabular features are useful for short-term station-level bicycle count forecasting.

The dashboard's planning indicators should be interpreted as screening tools. A high priority score does not automatically mean that infrastructure should be built or expanded. It means that the station deserves closer investigation because it combines high demand, peak pressure, recent growth, reliable forecasts, or good data coverage. Similarly, a low data coverage warning does not mean a location is unimportant; it means the evidence is not reliable enough for immediate planning conclusions.

9.1 Response to Proposal Feedback

The teaching feedback emphasized that the initial dashboard idea was too limited for infrastructure planning and that the project risked becoming a one-off exercise. Table 8 summarizes how the implemented version addresses these concerns.

Feedback point	**Implemented response**
Dashboard too limited for infrastructure planning	Added Planning Insights with demand, peak pressure, growth, coverage, forecast reliability, priority scores, and recommendation labels.
Risk of one-off analysis	Added preprocessing, forecast training, orchestration scripts, JSON run summaries, and generated artifacts consumed by the dashboard.
Need clearer retraining and evaluation over time	Added rolling backtests and command-line pipeline refreshes for monthly data updates.
Baseline versus Prophet too standard	Added histogram gradient boosting as a feature-based scikit-learn model and compared all models against the seasonal naive baseline.
Need stronger decision-making value	Added infrastructure priority shortlist and interpretation logic that distinguishes capacity candidates, growth signals, peak pressure, and data quality checks.

Table 8: How the current implementation responds to the proposal feedback.

10 Limitations and Future Work

The current system has several limitations. First, saved advanced forecasts are generated for the 12 busiest eligible stations by default. This is sufficient for a responsive demonstration, but a full production deployment should run forecasts for all eligible stations or allow scheduled background training. The `-all-stations` option already supports this extension.

Second, the model uses only historical bicycle count data. Weather, holidays, school vacation periods, road works, and local events may explain additional variation. These external data sources could improve forecasting and planning interpretation, but they would also introduce new data engineering challenges such as time alignment, missing values, and source reliability. For the current version, we prioritize a robust and reproducible internal-data pipeline.

Third, the priority score is a heuristic ranking formula. Its weights are chosen to reflect planning intuition: demand and peak pressure are most important, growth indicates emerging needs, forecast reliability supports short-term monitoring, and coverage protects against poor data

quality. In a real policy setting, these weights should be validated with domain experts and possibly adapted by municipality or infrastructure type.

Fourth, the Dockerfile has been prepared but could not be fully verified on the current machine because the Docker CLI was not available during the latest project check. Therefore, the correct claim is that the project is container-ready, not that it has been fully container-tested. A future deployment could install Docker Desktop, build the image, mount `data/processed/`, and schedule monthly pipeline refreshes when AWV releases new CSV files.

Finally, the current GitHub-ready structure separates source code, report files, dashboard files, and generated artifacts. The large raw data folder and processed Parquet outputs are ignored by Git. This keeps the repository lightweight while allowing results to be regenerated from the documented pipeline commands.

11 Conclusion

This project delivers a working station-level bicycle traffic monitoring, forecasting, and planning-support dashboard for Flanders. It processes more than 5.3 million hourly station records from 150 stations, trains and evaluates short-term forecasting models for selected high-demand stations, and exposes the full workflow through a Shiny dashboard.

The most important contribution is not only the dashboard interface, but the complete analytics workflow behind it. The system supports ingestion, preprocessing, artifact storage, rolling model evaluation, future forecast generation, and transparent dashboard reporting. This makes the project closer to a modern data analytics system than a one-off analysis.

From a planning perspective, the dashboard helps identify high-demand stations, peak pressure points, fast growth locations, data quality risks, and predictable stations. From an ML engineering perspective, it demonstrates repeatable pipelines, baseline comparison, rolling backtesting, and model selection based on WAPE. These elements directly address the proposal feedback and align with the Modern Data Analytics course themes of Python data science, machine learning pipelines, dashboarding, versioning, and analytics infrastructure.

A Reproducibility Commands

The processed data and model artifacts are ignored by Git, but they can be regenerated locally from the downloaded raw CSV files.

```
python codes/run_pipeline.py \
  --start-month 2019-08 \
  --end-month 2026-04 \
  --top-stations 12 \
  --backtest-windows 3
```

If processed Parquet files already exist and only forecast artifacts need to be refreshed:

```
python codes/run_pipeline.py \
  --skip-preprocess \
  --top-stations 12 \
  --backtest-windows 3
```

To run the dashboard locally:

```
shiny run --host 127.0.0.1 --port 8000 app/app.py
```

B Main Project Files

File	Purpose
`codes/preprocess.py`	Builds hourly counts and station summaries from raw CSV files.
`codes/train_forecasts.py`	Runs rolling backtests and generates future forecasts.
`codes/run_pipeline.py`	Orchestrates preprocessing and forecast training.
`app/app.py`	Shiny dashboard application.
`app/www/styles.css`	Dashboard styling.
`Dockerfile`	Container definition for the dashboard.
`DASHBOARD_README.md`	Local running instructions.

C Dashboard Demonstration Checklist

1. Start with the dashboard purpose: station-level monitoring, forecasting, and planning support.
2. Show **Overview**: map, total cyclists, station rankings, and growth rankings.
3. Show **Station Detail**: one busy station, daily pattern, hour-of-day pattern, weekday profile, and coverage.
4. Show **Forecast**: three models, rolling backtests, WAPE comparison, and future 168-hour forecast.
5. Show **Planning Insights**: priority score formula and recommendation categories.
6. Show **ML Engineering**: pipeline status, generated artifacts, and re-run commands.
7. End with limitations: default forecasts are trained for 12 high-demand stations; all-station training is supported but slower; weather and holidays are future extensions.

D Docker Verification Note

The project includes a `Dockerfile` and `.dockerignore`. The intended verification commands are:

```
docker build -t mda-bike-dashboard .
docker run --rm -p 8000:8000 \
  -v "$PWD/data/processed:/app/data/processed" \
  mda-bike-dashboard
```

During the latest local check, the command `docker -version` returned `command not found`. Therefore, Docker build and run were not executed on this machine. This should be verified after installing Docker Desktop or another Docker runtime.